C++ Project / Planning

Trivia Quiz System

Student 1	Ion Popescu	**Student 2**	Maria Ionescu
Project	Trivia Quiz System	**Language**	C++17

I. Task Description

Application 1 (main) — Player App — Student 1: Ion Popescu

Responsible for the player-facing side of the trivia game:

- Play the quiz by answering questions loaded from questions.txt
- Save mid-game progress to progress.txt and resume later
- Save final scores to leaderboard.txt after finishing
- View the leaderboard ranked by score
- Reset own saved progress via command-line

Application 2 (admin) — Admin App — Student 2: Maria Ionescu

Responsible for the administrative side:

- Add new trivia questions to questions.txt
- Delete or view questions by number
- View the leaderboard written by App 1 (bidirectional flow)
- Delete specific players from the leaderboard
- Clear the entire leaderboard
- View and reset player progress written by App 1

Bidirectional flow: App 1 writes leaderboard.txt and progress.txt; App 2 reads them. App 2 writes questions.txt; App 1 reads it.

II. Data Structures (Classes)

Class: Question

Stores a trivia question with four options, the correct answer index, and difficulty. Overloads operator<< for display.

text	std::string	The question text
options[4]	std::string	The four answer choices

correct	int	Index of correct answer (1–4)
difficulty	int	1=Easy, 2=Medium, 3=Hard
difficultyLabel()	std::string	Returns human-readable difficulty
operator<<	friend ostream&	Overloaded stream output operator

Class: Player (base class)

Holds a player's name and score. Base class for GamePlayer. Overloads operator<< for display.

name	std::string	Player's name
score	int	Current score
getName() / getScore()	accessors	Return name and score
saveToLeaderboard()	virtual void	Appends score to leaderboard.txt
operator<<	friend ostream&	Overloaded stream output

Class: GamePlayer (inherits Player)

Extends Player with progress save/load functionality. **IS-A Player** (inheritance). **HAS-A vector<Question>** (association).

questionIndex	int	Current question position for save/resume
saveProgress()	void	Writes name, index, score to progress.txt
loadProgress()	bool	Reads saved progress from progress.txt
clearProgress()	static void	Resets progress.txt to 'none 0 0'

Class: User (base class) — admin.cpp

name	std::string	User's display name
getName()	std::string	Returns name

getRole()	virtual std::string	Returns role label (overridden in Admin)

Class: Admin (inherits User) — admin.cpp

IS-A User (inheritance). Adds authentication and elevated operations.

authenticate(pwd)	bool	Returns true if password matches 'admin123'
getRole()	std::string override	Returns 'Admin'

Template Function: displayList

Generic template function used in both apps to display any vector of objects that support operator<<. Satisfies the template requirement.

```
template <typename T>
void displayList(const std::vector<T>& items, const std::string& header)
```

III. File Structure

questions.txt

Written by App 2 (admin). Read by App 1 (main).

Format:

```
<question text>
<option 1>
<option 2>
<option 3>
<option 4>
<correct (1-4)>
<difficulty (1-3)>
[repeats for each question]
```

leaderboard.txt

Written by App 1 (main). Read by App 2 (admin).

Format:

```
<player_name> <score>
[one entry per line]
```

progress.txt

Written by App 1 (main). Read by App 2 (admin).

Format:

```
<player_name> <question_index> <score>
[single line, 'none 0 0' when empty]
```

IV. Commands the Apps Will Implement

Application 1 — main

./main play <name>	Start a new game or resume saved progress for player
./main leaderboard	Display all leaderboard entries, sorted by score descending
./main questions	Display all questions from questions.txt
./main progress	Show the current saved progress entry
./main reset_progress <name>	Clear saved progress for the named player

Application 2 — admin

./admin view_questions	Display all questions with numbers
./admin add_question <text> <o1> <o2> <o3> <o4> <correct> <diff>	Append a new question to questions.txt
./admin delete_question <number>	Remove question by its display number
./admin view_leaderboard	Read and display leaderboard.txt (written by App 1)
./admin delete_player <name>	Remove a player entry from leaderboard.txt
./admin clear_leaderboard	Empty leaderboard.txt completely
./admin view_progress	Read and display progress.txt (written by App 1)
./admin reset_progress	Reset progress.txt to 'none 0 0'
